Extraction Of the Dataset fro Yahoo Finance

```
import requests
import pandas as pd
import time

ticker = "GC=F"
start_date = "2020-11-16"
end_date = "2025-11-16"

period1 = int(time.mktime(pd.Timestamp(start_date).timetuple()))
period2 = int(time.mktime(pd.Timestamp(end_date).timetuple()))

# JSON API URL (never blocked)
url = (
    f"https://query1.finance.yahoo.com/v8/finance/chart/{ticker}"
    f"?period1={period1}&period2={period2}&interval=1d"
    f"&includePrePost=false&events=div,splits"
)

headers = {
    "User-Agent": "Mozilla/5.0"
}

response = requests.get(url, headers=headers).json()

# Extract data
result = response['chart']['result'][0]
timestamps = result['timestamp']
indicators = result['indicators']['quote'][0]
adjclose = result['indicators']['adjclose'][0]['adjclose']

# Create DataFrame
df = pd.DataFrame({
    "Date": pd.to_datetime(timestamps, unit='s'),
    "Open": indicators['open'],
    "High": indicators['high'],
    "Low": indicators['low'],
    "Close": indicators['close'],
    "Adj Close": adjclose,
    "Volume": indicators['volume'],
})

# Save
df.to_csv("gold_futures_full_history.csv", index=False)

df.head()
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	2020-11-16 05:00:00	1874.599976	1887.300049	1871.000000	1887.300049	1887.300049	6.0
1	2020-11-17 05:00:00	1888.400024	1888.400024	1884.500000	1884.500000	1884.500000	59.0
2	2020-11-18 05:00:00	1873.500000	1873.500000	1873.500000	1873.500000	1873.500000	152.0
3	2020-11-19 05:00:00	1865.800049	1865.800049	1856.000000	1861.099976	1861.099976	59.0
4	2020-11-20 05:00:00	1871.199951	1872.599976	1871.199951	1872.599976	1872.599976	1900.0

Choosing the variable to be predicted

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import tensorflow as tf
import joblib

SEQ_LEN = 60
FEATURES = ["Close"]
```

Loading the Dataset and Dropping Nulls if any

```
df = pd.read_csv("gold_futures_full_history.csv")
```

```
if "Date" in df.columns:
    df["Date"] = pd.to_datetime(df["Date"])
    df = df.sort_values("Date").reset_index(drop=True)

df = df.dropna(subset=FEATURES).reset_index(drop=True)
```

```
data = df[FEATURES].astype("float32").values   # shape (N,1)
print(data.shape)
```

```
(1258, 1)
```

Min-Max(Scaling)

```
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)
joblib.dump(scaler, "scaler_minmax.save")
```

```
['scaler_minmax.save']
```

Taking a Sequence window of 60 Days

```
def create_sequences(arr, seq_len=SEQ_LEN):
    X, y = [], []
    for i in range(seq_len, len(arr)):
        X.append(arr[i-seq_len:i])
        y.append(arr[i])      # next day Close
    return np.array(X), np.array(y)

X, y = create_sequences(data_scaled, SEQ_LEN)
print(X.shape, y.shape)
```

```
(1198, 60, 1) (1198, 1)
```

```
total = len(X)
train_end = int(total * 0.7)
val_end   = int(total * 0.85)

X_train, y_train = X[:train_end], y[:train_end]
X_val,   y_val   = X[train_end:val_end], y[train_end:val_end]
X_test,  y_test  = X[val_end:], y[val_end:]
```

```
BATCH = 32

train_ds = tf.data.Dataset.from_tensor_slices((X_train, y_train)).batch(BATCH).prefetch(tf.data.AUTOTUNE)
val_ds   = tf.data.Dataset.from_tensor_slices((X_val, y_val)).batch(BATCH).prefetch(tf.data.AUTOTUNE)
test_ds  = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(BATCH).prefetch(tf.data.AUTOTUNE)
```

Building the RNN Model

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

num_features = X_train.shape[-1]   # 1 for univariate

model = Sequential([
    SimpleRNN(128, return_sequences=True, input_shape=(SEQ_LEN, num_features)),
    Dropout(0.2),

    SimpleRNN(64),
    Dropout(0.2),

    Dense(num_features)   # output one close value
])

model.compile(
    loss='mse',
    optimizer=Adam(0.001),
    metrics=['mae']
```

```
)

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape
  super().__init__(**kwargs)
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
simple_rnn_1 (SimpleRNN)	(None, 60, 128)	16,640
dropout (Dropout)	(None, 60, 128)	0
simple_rnn_2 (SimpleRNN)	(None, 64)	12,352
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 1)	65

Total params: 29,057 (113.50 KB)
Trainable params: 29,057 (113.50 KB)
Non-trainable params: 0 (0.00 B)

Training the Model

```
early_stop = EarlyStopping(monitor='val_loss', patience=6, restore_best_weights=True)
lr_sched = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=80,
    callbacks=[early_stop, lr_sched],
    verbose=1
)
```

```
Epoch 1/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 8s 149ms/step - loss: 0.2505 - mae: 0.3838 - val_loss: 0.0026 - val_mae: 0.0428 - lear
Epoch 2/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0106 - mae: 0.0817 - val_loss: 8.9007e-04 - val_mae: 0.0237 - l
Epoch 3/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0045 - mae: 0.0544 - val_loss: 4.3877e-04 - val_mae: 0.0163 - l
Epoch 4/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0040 - mae: 0.0509 - val_loss: 5.2224e-04 - val_mae: 0.0185 - l
Epoch 5/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0032 - mae: 0.0447 - val_loss: 5.2801e-04 - val_mae: 0.0184 - l
Epoch 6/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0025 - mae: 0.0399 - val_loss: 4.9753e-04 - val_mae: 0.0179 - l
Epoch 7/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0022 - mae: 0.0377 - val_loss: 3.1677e-04 - val_mae: 0.0139 - l
Epoch 8/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0020 - mae: 0.0362 - val_loss: 3.8791e-04 - val_mae: 0.0159 - l
Epoch 9/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0016 - mae: 0.0321 - val_loss: 4.9174e-04 - val_mae: 0.0181 - l
Epoch 10/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0014 - mae: 0.0287 - val_loss: 4.5132e-04 - val_mae: 0.0174 - l
Epoch 11/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0017 - mae: 0.0323 - val_loss: 2.8395e-04 - val_mae: 0.0133 - l
Epoch 12/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0015 - mae: 0.0305 - val_loss: 0.0015 - val_mae: 0.0343 - learn
Epoch 13/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0013 - mae: 0.0274 - val_loss: 0.0010 - val_mae: 0.0276 - learn
Epoch 14/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0011 - mae: 0.0261 - val_loss: 4.1003e-04 - val_mae: 0.0165 - l
Epoch 15/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0012 - mae: 0.0267 - val_loss: 2.7750e-04 - val_mae: 0.0133 - l
Epoch 16/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0014 - mae: 0.0299 - val_loss: 3.1020e-04 - val_mae: 0.0142 - l
Epoch 17/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0011 - mae: 0.0266 - val_loss: 2.9049e-04 - val_mae: 0.0137 - l
Epoch 18/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0011 - mae: 0.0268 - val_loss: 3.3124e-04 - val_mae: 0.0147 - l
Epoch 19/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0012 - mae: 0.0264 - val_loss: 3.8409e-04 - val_mae: 0.0159 - l
Epoch 20/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0011 - mae: 0.0262 - val_loss: 4.5444e-04 - val_mae: 0.0174 - l
Epoch 21/80
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 9.0512e-04 - mae: 0.0235 - val_loss: 3.6836e-04 - val_mae: 0.0156
```

Prediction using the Model

```
pred_scaled = model.predict(test_ds)
pred_scaled = pred_scaled.reshape(-1, num_features)
```

```
WARNING:tensorflow:5 out of the last 13 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_s
6/6 ━━━━━━━━━━━━━━━━━━━━ 1s 97ms/step
```

Inverse Transform the values to original Values

```
pred = scaler.inverse_transform(pred_scaled)
y_test_orig = scaler.inverse_transform(y_test)
```

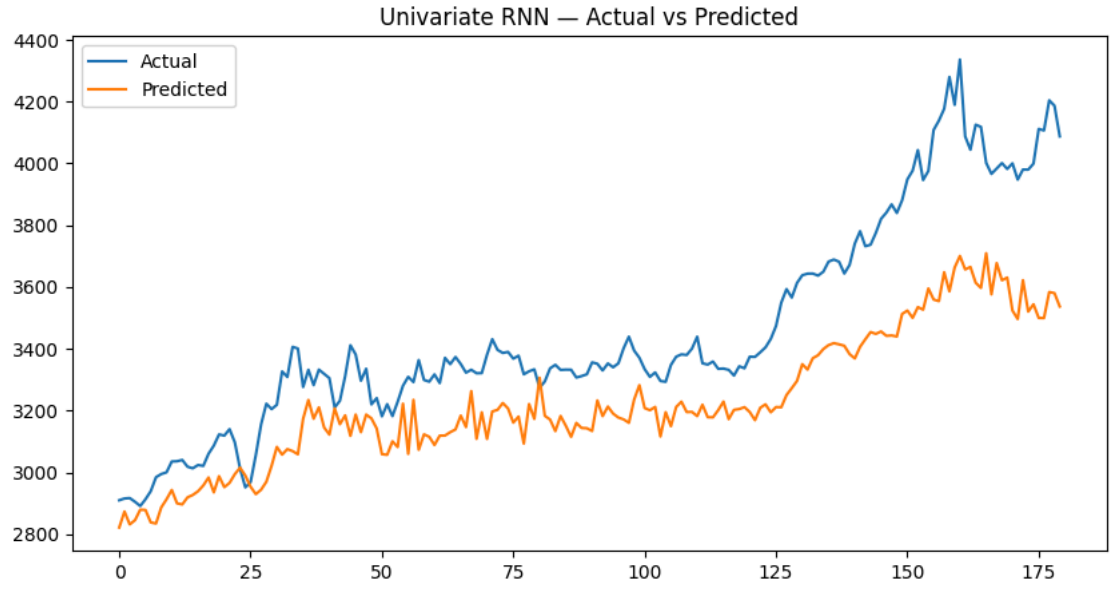
Metrics for meeasuring the performance of the model

```
from sklearn.metrics import mean_squared_error, mean_absolute_error

rmse = np.sqrt(mean_squared_error(y_test_orig, pred))
mae = mean_absolute_error(y_test_orig, pred)
mape = np.mean(np.abs((y_test_orig - pred) / y_test_orig)) * 100

print("RNN RMSE:", rmse)
print("RNN MAE:", mae)
print("RNN MAPE:", mape)
```

```
RNN RMSE: 272.069384532696
RNN MAE: 229.83074951171875
RNN MAPE: 6.356969
```

Graph-Actual Vs Predicted

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
plt.plot(y_test_orig, label='Actual')
plt.plot(pred, label='Predicted')
plt.title("Univariate RNN — Actual vs Predicted")
plt.legend()
plt.show()
```



Business Insights:

1.RMSE (272) and MAE (230) are relatively high → your model makes some large absolute errors.

2.MAPE (6.36%) is good → percentage-wise the predictions are accurate.

This usually means your target values are large in scale, so absolute errors look big even though the model captures the trend well.

To improve: try LSTM/GRU instead of RNN, add more lag/rolling features, or tune hyperparameters.